

Relaciones de Clase y Herencia

Este apunte de estudio profundiza en los pilares fundamentales de la programación orientada a objetos (POO) utilizando el lenguaje Java, centrándose en la arquitectura de clases, la gestión de datos dinámicos y la organización profesional del código.

1. Relaciones entre Clases y Objetos como Atributos

En el diseño de software, las clases no existen de forma aislada; se comunican y colaboran entre sí para resolver problemas complejos. Comprender cómo una clase se relaciona con otra es vital para respetar principios de diseño como SOLID, que aboga por separar responsabilidades en componentes más pequeños y reutilizables.

Asociación: La interacción binaria

La asociación es la forma más básica de relación entre dos clases. Describe una actividad o vínculo donde una clase utiliza a otra. Se define como una relación binaria que describe una acción.

- **Ejemplo conceptual:** Un Estudiante toma un Curso. Aquí, hay una asociación entre la clase Estudiante y la clase Curso.
- **Implementación:** En Java, esto se traduce habitualmente en que una clase tiene un atributo que hace referencia a la otra clase, o bien posee métodos que reciben objetos de la otra clase como parámetros.

```
public class Estudiante {  
    private Curso[] cursos; // El estudiante sabe de sus cursos  
    public void agregarCurso(Curso c) {  
        // Lógica para asociar el curso  
    }  
}
```

Agregación: La relación "Todo-Parte" débil

La agregación es una forma especial de **asociación** que representa una relación de propiedad de tipo "todo-parte". Se considera una relación **débil** porque las partes pueden existir independientemente del objeto principal (el "todo").

- **Concepto clave:** El objeto principal "tiene" otros objetos, pero si el objeto principal se destruye, las partes no necesariamente desaparecen.

- **Ejemplo:** La estructura de datos tipo `Árbol` y sus `Nodos`. Un árbol puede estar vacío (sin nodos) y seguir siendo conceptualmente un árbol; asimismo, un nodo puede nacer sin tener hijos (nodos izquierdo o derecho).
- **Implementación en Java:** Los atributos no se inicializan necesariamente en el constructor de "todo" el `Árbol`, permitiendo que se añadan o eliminen dinámicamente en otro momento.

Composición: La relación "Todo-Parte" fuerte

La composición es una forma de agregación fuerte donde un objeto es propiedad **exclusiva** de otro. Es una relación de dependencia vital: si el objeto principal desaparece, sus partes constituyentes pierden su razón de ser o también desaparecen.

- **Escenario de uso:** Se prefiere la composición cuando las partes son obligatorias para que el objeto principal funcione o tenga sentido.
- **Ejemplo:** Un `Círculo` está compuesto por un `Punto` central y un `Punto` en su circunferencia. Sin estos puntos, el círculo no puede definirse, dibujarse ni calcular su área.
- **Implementación:** El objeto principal suele crear o recibir sus componentes en el momento de su instanciación (nace con ellos).

```
public class Circulo {
    private Punto centro; // El círculo "está compuesto" por puntos
    private Punto puntoP;

    public Circulo(Punto c, Punto p) {
        this.centro = c; // El círculo nace con sus puntos definidos
        this.puntoP = p;
    }
}
```

Uso de objetos como atributos

Tanto la agregación como la composición se mapean en código como **atributos** de clase. En lugar de utilizar solo tipos primitivos (como `int` o `double`), una clase utiliza otras clases como tipos de datos para sus campos o atributos.

Este enfoque favorece la **reutilización de código**. Por ejemplo, si tenemos una clase `Direccion`, tanto la clase `Persona` como la clase `Empresa` pueden incluir un atributo de tipo `Direccion`. Esto evita repetir campos como "calle" o "número" en múltiples lugares, delegando esa responsabilidad a una clase especializada para este tema.

2. Herencia: Jerarquías y Reutilización

La herencia es un paradigma de la POO que permite crear nuevas clases basadas en clases existentes, heredando sus características (atributos y métodos).

Jerarquías de clases: La relación "Es Un"

A diferencia de la composición ("tiene un"), la herencia se basa en una relación jerárquica de **"es un"**. Se utiliza para categorizar objetos desde lo más abstracto a lo más concreto.

- **Superclase (Madre):** Es la clase base, la más abstracta de la cadena. En Java, la clase raíz de todas las jerarquías es la clase `Object`, que define comportamientos comunes para todos los objetos de Java.
- **Subclase (Hija):** Es la clase que extiende a la superclase, heredando sus rasgos y añadiendo comportamientos más específicos.
- **Ejemplo:** Un `Deportista` **es una** `Persona`. Una `Persona` tiene nombre y edad; un `Deportista` hereda eso y añade por ejemplo el "deporte que practica".

Reutilización de código y especialización

La principal ventaja de la herencia es que evita la duplicación de código. Una subclase hereda automáticamente todos los miembros **públicos y protegidos** de su clase madre.

- **Herencia única:** En Java, una clase solo puede tener una clase ancestral directa (una sola madre), aunque puede tener muchas clases "hijas".
- **Herencia multinivel:** Se pueden crear cadenas de herencia (Clase A -> Clase B -> Clase C).
- **Especialización:** La subclase puede usar los métodos heredados tal cual, o bien reemplazarlos bajo el mismo nombre, o bien añadir nuevos métodos y atributos que la clase madre no posee.

3. Dinámica: Sobrescritura y Polimorfismo

La herencia no solo trata de copiar campos o atributos; permite que las subclases adapten de forma dinámica el comportamiento heredado.

Sobrescritura de métodos (Method Overriding)

Ocurre cuando una subclase proporciona su propia implementación de un método que ya existe en la superclase.

- **Reglas:** El método debe tener el **mismo nombre, mismo tipo de retorno y misma lista de parámetros** que el original.
- **Anotación @Override:** Se utiliza para indicar explícitamente al compilador que se está redefiniendo un método. Ayuda a detectar errores si la firma no coincide exactamente.
- **Visibilidad:** La visibilidad en la subclase no puede ser menor que en la superclase (por ejemplo, si en la madre es `protected`, en la hija no puede ser `private`).

Uso de referencias polimórficas

El polimorfismo es la capacidad de un objeto de comportarse de manera diferente según su tipo real en tiempo de ejecución, aunque se acceda a él a través de una referencia de su superclase.

- **Upcasting:** Es posible inicializar un objeto de una clase hija usando una referencia de la clase madre, por ej.: `Animal miMascota = new Perro();`
- **Ejecución dinámica:** Si se llama a un método que ha sido sobrescrito en la clase hija, Java ejecutará la versión de la clase hija, no la de la madre. Si el método no está sobrescrito, se llamará automáticamente al de la clase madre.
- **La palabra clave super:** Se usa dentro de un método sobrescrito para invocar la lógica original de la madre antes o después de añadir la lógica específica de la hija.

4. Colecciones y Estructuras Dinámicas

Java proporciona el **Collections Framework** para manejar conjuntos de datos de forma más eficiente y flexible que los arrays tradicionales, los cuales tienen un tamaño fijo.

List (Listas)

Representan un **conjunto ordenado** de elementos donde cada uno tiene una posición específica o índice (empezando en 0).

- **ArrayList:** Basada en arrays internos; es rápida para acceso aleatorio pero lenta si se añaden/quitan elementos constantemente en posiciones intermedias.
- **LinkedList:** Basada en nodos enlazados; es ideal para listados muy variables donde se insertan y eliminan elementos con frecuencia.

- **Métodos comunes a ambos tipos:** `add(elemento)`, `get(indice)`, `remove(indice)`, `set(indice, valor)` y `size()`.

Set (Conjuntos)

Son colecciones de elementos **desordenados** que no permiten duplicados.

- **Características:** No existe el concepto de índice o posición. Son útiles principalmente para verificar si un elemento "pertenece" o no al grupo.
- **Implementaciones:** `HashSet` (más común) y `TreeSet`.
- **Métodos comunes:** `add(elemento)`, `contains(elemento)`, `remove(elemento)`.

Map (Mapas)

Estructuras que relacionan una **clave única** con un **valor**. Funcionan de forma similar a un diccionario o a un archivo JSON.

- **Clave-Valor:** Se usa la clave para acceder directamente al valor asociado.
- **Implementación:** `HashMap` es la más habitual.
- **Métodos comunes:** `put(clave, valor)` para añadir y `get(clave)` para recuperar.

Iteración sobre colecciones

Para recorrer estas estructuras de forma segura, Java utiliza la interfaz **Iterable** y el objeto **Iterator**.

- **Iterator:** Es como un "clon" o cursor que permite recorrer la colección elemento por elemento sin modificar la estructura original durante el recorrido (evitando errores de concurrencia).
- **Métodos del Iterator:** `hasNext()` (¿hay más?), `next()` (devuelve el siguiente) y `remove()` para borrar.
- **Bucle For-each:** Es una forma simplificada y elegante de usar iteradores internamente para recorrer cualquier clase que implemente `Iterable`.

5. Organización Modular: Paquetes (Packages)

Los paquetes son contenedores que agrupan clases e interfaces relacionadas, funcionando de manera similar a las carpetas (o subdirectorios) en un sistema de archivos.

Propósito y Estructura

La organización en paquetes es crucial para la **mantenibilidad y escalabilidad** de un proyecto.

- **Encapsulamiento:** Ayudan a prevenir conflictos de nombres; dos clases pueden llamarse igual si están en paquetes distintos.
- **Nomenclatura:** Se recomienda usar el dominio inverso de la empresa (ej. `com.empresa.proyecto.modulo`) para garantizar la unicidad global.
- **Jerarquía física:** Un paquete `entradas.usuarios` se traduce físicamente en una estructura de carpetas `entradas/usuarios/` en el disco duro.

Visibilidad e Importación

La pertenencia a un paquete afecta directamente a quién puede ver y usar una clase.

- **Sentencia `package`:** Debe ser la primera línea del archivo fuente de la clase, por ejemplo: `package mi.paquete;`
- **Visibilidad:** Para que una clase sea accesible fuera de su paquete, debe ser declarada como `public`. Si no tiene modificador, solo es visible dentro de su propio paquete.
- **Sentencia `import`:** Permite usar clases definidas en otros paquetes. Se puede importar una clase específica (`import paquete.Clase;`) o todo un paquete (`import paquete.*;`).
- **Separación de responsabilidades:** Una buena práctica es separar las clases según su función, por ejemplo, tener paquetes distintos para la lógica de negocio (`domain`), el acceso a datos (`dao`) y la interfaz de usuario (`view`).